

Claves foráneas e integridad referencial

Las tablas de Tienda Latam ya están creadas con sus llaves primarias y restricciones. Pero en este momento, cada tabla existe por separado. Nada le impide a alguien registrar un pedido con un cliente que no existe, o asignar un producto a una categoría que nunca se creó. El motor lo acepta, el dato queda guardado y el error no se descubre hasta que alguien corre un reporte y los números no cierran.

Las llaves foráneas existen exactamente para evitar eso. Son la instrucción que le dice al motor: "este campo debe coincidir con un valor que ya existe en otra tabla". Si no coincide, el motor rechaza el dato antes de que entre.

Tablas padre y tablas hija

Antes de escribir código, hay un concepto que hace todo más claro.

Cuando una tabla referencia a otra, la que tiene la llave primaria es la **tabla padre** y la que la referencia es la **tabla hija**. La tabla hija no puede tener un valor en su llave foránea que no exista previamente en la tabla padre.

Piénsalo así: no puede haber un empleado asignado a una sucursal que no existe. La sucursal tiene que estar creada primero. Sucursales es la tabla padre; empleados es la tabla hija.

Eso también determina el orden en que debes crear las tablas: **siempre primero las padres, luego las hijas**.

Cómo se escribe una llave foránea

La palabra clave es **REFERENCES**. Después de definir el tipo de dato del campo, le dices a qué tabla y a qué columna hace referencia.

```
SQL
CREATE TABLE sucursales (
    sucursal_id INTEGER GENERATED ALWAYS AS IDENTITY
    PRIMARY KEY,
```

```

    nombre      TEXT      NOT NULL,
    ciudad      TEXT      NOT NULL,
    pais_id     INTEGER   NOT NULL REFERENCES
países(pais_id)
);

```

La línea `pais_id INTEGER NOT NULL REFERENCES países(pais_id)` se lee así: este campo es un entero, es obligatorio, y su valor debe existir en la columna `pais_id` de la tabla `países`. Si alguien intenta insertar una sucursal con un `pais_id` que no está en la tabla de países, el motor lo rechaza.

Una tabla puede tener múltiples llaves foráneas

No hay límite. Una tabla puede referenciar a tantas tablas padre como necesite. El caso más claro en Tienda Latam es la tabla de pedidos:

```

SQL
CREATE TABLE pedidos (
    pedido_id    INTEGER   GENERATED ALWAYS AS IDENTITY
PRIMARY KEY,
    fecha        DATE      NOT NULL DEFAULT CURRENT_DATE,
    cliente_id   INTEGER   NOT NULL REFERENCES
clientes(cliente_id),
    sucursal_id  INTEGER   NOT NULL REFERENCES
sucursales(sucursal_id),
    empleado_id  INTEGER   NOT NULL REFERENCES
empleados(empleado_id)
);

```

Un pedido siempre tiene un cliente, una sucursal y un empleado. Las tres llaves foráneas garantizan que los tres existan antes de que el pedido pueda registrarse. Si falta cualquiera de los tres, el motor no acepta el pedido.

La tabla intermedia: cuando la relación es de muchos a muchos

Ya vimos en la clase de normalización que entre pedidos y productos existe una relación de muchos a muchos: un pedido puede tener varios productos y un producto puede aparecer en varios pedidos. Eso se resuelve con una tabla intermedia que referencia a las dos.

```
SQL
CREATE TABLE detalle_pedidos (
    detalle_id INTEGER GENERATED ALWAYS AS IDENTITY
    PRIMARY KEY,
    pedido_id INTEGER NOT NULL REFERENCES
    pedidos(pedido_id),
    producto_id INTEGER NOT NULL REFERENCES
    productos(producto_id),
    cantidad INTEGER NOT NULL,
    precio NUMERIC(10,2) NOT NULL
);
```

Esta tabla no tiene datos propios del pedido ni del producto: solo los referencia. Lo que sí le pertenece a ella son la cantidad y el precio al momento de la venta, que como vimos antes pueden variar por descuentos o condiciones del mercado.

El orden de creación importa

Como las tablas hija dependen de que las padre existan primero, hay un orden lógico para construir el modelo completo de Tienda Latam:

Primero las tablas padre — no referencian a nadie:

1. Países
2. Tipo de cliente
3. Categorías

Luego las tablas que dependen de las anteriores:

4. Sucursales → referencia Países
5. Clientes → referencia Tipo de cliente
6. Productos → referencia Categorías
7. Empleados → referencia Sucursales

Por último las tablas que dependen de varias:

8. Pedidos → referencia Clientes, Sucursales, Empleados
9. Detalle de pedidos → referencia Pedidos, Productos

Si intentas crear una tabla hija antes que su padre, el motor devuelve un error porque la tabla referenciada todavía no existe.

Qué pasa cuando no usas llaves foráneas

Sin llaves foráneas, el motor no tiene forma de verificar que los datos sean consistentes entre tablas. Algunos ejemplos concretos de lo que puede pasar:

- Se registra un pedido con un `cliente_id` que no existe en la tabla de clientes. El pedido queda en la base, pero no hay forma de saber a quién pertenece.
- Se elimina un empleado que tiene pedidos asignados. Los pedidos quedan huérfanos, sin empleado responsable.
- Se inserta un producto con una categoría que nunca se creó. Los reportes por categoría van a ignorar ese producto o dar error.

Ninguno de estos errores produce un mensaje visible en el momento. Se descubren después, cuando alguien cruza los datos y algo no cierra. Las llaves foráneas los bloquean antes de que entren.

El modelo completo de Tienda Latam

Con las llaves foráneas definidas, el modelo que venías diseñando desde las primeras clases queda completamente materializado en código. Cada relación que

dibujaste en el diagrama tiene ahora su instrucción SQL correspondiente, y el motor es el encargado de hacerla cumplir.

Eso es la **integridad referencial**: la garantía de que los datos de tu base son consistentes entre sí, no porque alguien lo revise manualmente, sino porque el motor no permite que sean de otra forma.

Con la estructura completa y las relaciones definidas, el siguiente paso es empezar a trabajar con los datos: insertarlos, consultarlos y filtrarlos. Ahí es donde SQL empieza a mostrar todo su potencial.